



SDEV2171

Lesson 10: Lists and Dynamic Data Rendering

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain how collections of data can be rendered dynamically in a React Native UI
- Use arrays to represent collections of app data
- Render repeated UI from array data
- Implement a simple `FlatList`
- Use conditional rendering to show an alternate empty-state message

Lesson Roadmap

1. Reconnect lesson 09 state to repeated list data
2. Define arrays, repeated UI, and empty-state rendering
3. Compare `map` and `FlatList`
4. Guided build of one list screen
5. Testing and troubleshooting
6. Checkpoint and lesson-11 handoff

Starting Point From Lesson 09

- use the same Expo Router project from lesson 09
- confirm the home screen still opens the detail screen
- confirm the detail screen still updates based on state

If your project is not ready

- if your project is `configured` , keep using your own project
- if your project is `partial` or `blocked` , follow your recorded remediation steps first
- if recovery will take too long, use the instructor-guided reference path and record your next step

Why Lists Matter

- many app screens show repeated items instead of one value
- a shopping list, message list, task list, or feed is still just data plus UI rules
- lesson 09 taught one value changing on screen
- lesson 10 teaches many values rendered with one repeatable pattern

Arrays As Repeated Data

- an array stores multiple items in order
- each item can describe one repeated UI row
- when the array changes, the list UI can change too

Example data shape

```
const [studySteps, setStudySteps] = useState([
  { id: '1', label: 'Charge laptop' },
  { id: '2', label: 'Pack notebook' },
  { id: '3', label: 'Review key terms' },
]);
```

Two Rendering Ideas Today

Tool

Job

`map`

helps explain repeated UI from array data

`FlatList`

built-in React Native list component for the main implementation

Teaching rule

- understand the repetition pattern first
- then use `FlatList` for the working lesson screen

Conditional Rendering For Empty States

- a list screen should not become confusing when the array is empty
- if there are items, show the list
- if there are no items, show a clear message instead

Plain-language rule

- one condition
- two possible UI results

Quick Sort Before You Edit

Values that should stay in state

- the current draft text
- the array of list items

UI that should be rendered from that state

- one repeated row for each item
- one empty-state message if the array has no items

Decision rule

- if the value changes, keep it in state
- if the UI is only showing what the state already contains, render it from that state

Minimum Repetition With `map`

```
studySteps.map((step) => (  
  <Text key={step.id}>{step.label}</Text>  
))
```

What this means

- each array item becomes one UI element
- `key` helps React track repeated items
- the list output depends on the current array data



ESSENTIAL
TO ALBERTA



Main Lesson Pattern With FlatList

```
<FlatList
  data={studySteps}
  keyExtractor={({item}) => item.id}
  renderItem={({ item }) => <Text>{item.label}</Text>}
/>
```

What this means

- `data` supplies the array
- `renderItem` explains one repeated row
- `keyExtractor` gives each row a stable identity

Empty State Pattern

```
{studySteps.length === 0 ? (  
  <Text>No study steps yet. Add one to build the list.</Text>  
) : (  
  <FlatList ... />  
)}
```

What to notice

- the condition checks the array length
- one branch shows the message
- the other branch shows the list

Guided Build Order

1. confirm the lesson-09 baseline
2. open `src/app/details.js`
3. remove the old single preview target from lesson 09
4. replace it with array-shaped state
5. test one hard-coded repeated rendering pass
6. switch the main screen to `FlatList`
7. add one empty-state message
8. test adding and clearing items
9. keep the clearest working version

Lesson 09 To Lesson 10 Conversion

Lesson 09 screen

- one input-driven screen
- one single preview target or child preview card
- one visible response area

Lesson 10 screen

- one input area for `draftStep`
- one array holding repeated items
- one list when items exist
- one empty-state message when no items exist

Replace rule

- keep any useful input control
- remove the old single preview output
- replace it with list output plus an empty-state branch

Add And Clear Actions

```
const [draftStep, setDraftStep] = useState('');

function handleAddStep() {
  const trimmedStep = draftStep.trim();

  if (trimmedStep.length === 0) return;

  setStudySteps((currentSteps) => [
    ...currentSteps,
    { id: String(Date.now()), label: trimmedStep },
  ]);
  setDraftStep('');
}

function handleClearSteps() {
  setStudySteps([]);
}
```

- one action adds an item into the array
- one action clears the array
- the screen changes because the array state changed

Extraction-Free Lesson Boundary

- this lesson is about repeated data and repeated UI
- you do not need new child components to succeed
- keep the repeated row simple
- avoid advanced list interaction or multiple list screens

Visible Target For Today

- the app still opens the home and detail routes correctly
- the detail screen stores repeated items in an array
- the screen renders a visible list when items exist
- the screen renders a clear empty-state message when items do not exist
- the back path still works

Common List Mistakes

- trying to render the whole array directly instead of one item at a time
- forgetting to pass the array into `FlatList`
- forgetting `renderItem`
- using a weak or missing item key
- clearing the list without showing an empty-state message
- changing too many things before retesting

Recovery Path If The Build Is Blocked

- `configured` : the list renders and the empty state appears correctly
- `partial` : the list or empty-state flow exists, but one part still fails
- `blocked` : the list-rendering flow cannot be completed during class

If you are blocked

- compare your own `src/app/details.js` to the reference app
- repair one part at a time in this order:
 - first: array state
 - second: `renderItem`
 - third: `FlatList`
 - fourth: empty-state condition

Checkpoint

Students should be ready to show

- one array storing repeated items
- one visible list rendered from that array
- one empty-state message that appears when the array is cleared
- one action that adds an item and one action that clears the list

Students should also be able to answer

- Which state value stores the repeated items?
- What does `renderItem` do?
- What condition switches the screen from list view to empty-state view?

What We Learned Today

- repeated UI starts with repeated data
- arrays are a simple way to store repeated items
- `FlatList` is the main React Native tool for flat list rendering
- conditional rendering helps the screen stay clear when no items exist
- this prepares us for debugging dynamic data and later fetching remote data

Next Step

- keep the lesson-10 list screen in a known-good state
- make sure the list and empty state both still work after edits
- be ready to debug dynamic UI behaviour in lesson 11